Chapter 3
Optimization Algorithms

Contents of this Chapter

- The role of optimization in Machine learning and deep learnin
  - √ Mathematical formulation and update rule
- Gradient decent (GD)
  - √ Bath gradient decent (BGD)
  - √ Stochastic gradient decent(SGD)
  - √ Minibath gradient decent (MBGD)
- Varients of SGD
  - √ SGD with momentum
  - √ Nesterov Accelerated Gradient (NAG)

Contents (Con...)

- Adaptive Gradient Methods
  - √ AdaGrad
  - √ RMSProp
- Adam Optimizer and Modern Variants
  - √ Adam
- Learning Rate Schedule

Learning Objective

By the end of this chapter, students will be able to:

- Explain the role of optimization algorithms in training machine learni and deep learning models
- Differentiate between major optimization approaches, including Batch Gradient Descent, Stochastic Gradient Descent (SGD), and Mini-Batch Gradient Descent.
- Describe the mathematical foundations of gradient-based optimization, including gradients, learning rate, convergence, and cost landscape.

Con...

- Analyze the strengths and limitations of common optimization algorithms such as SGD, Momentum, Nesterov, AdaGrad, RMSProp, Adam, AdamW, and others.

- Select an appropriate optimization algorithm for different machine learning tasks based on dataset size, model complexity,
  and convergence behavior.

- Evaluate the performance and convergence behavior of different optimizers using metrics such as loss curves, accuracy,
  and computational efficiency.

The role of optimization in ML/DL

- Optimization is at the core of machine learning (ML) and deep learning (DL).
- It determines how models learn, how fast they improve errors, and how well they generalize.
- It is used to minimize the loss function
  - Machine learning models learn by reducing the difference between the model's predictions and the true values
- This difference is quantified using a loss function (e.g., MSE, cross-entropy).
- Optimization deal with to find optimal model parameters (weights, biases)

The role of optimization in ML/DL

- Optimization is the engine that powers learning in ML and DL.
  - Updates model parameters
  - Minimizes loss
  - Enables deep network training
  - Controls convergence speed
  - Improves generalization
  - Supports large-scale training

Objective Function / Loss Function

- The objective function (also called loss function or cost function) $J(\theta)$
- General Form:

  For supervised learning:

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m} L(f_\theta(x^i), y^i))$$

where, m is number of training examples

$L(.)$ loss for one example

$f_\theta(x^i)$ is a model prediction

y^i, true lable

θ is all model parameter

The role of optimization in ML/DL

- Optimization finds the model parameters (weights, biases) that:

$$\theta^* = argmin_\theta J(\theta)$$

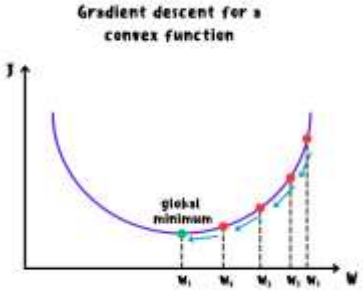
- We want to find the value of θ (model parameters) that gives the smallest possible value of the loss function J(θ).

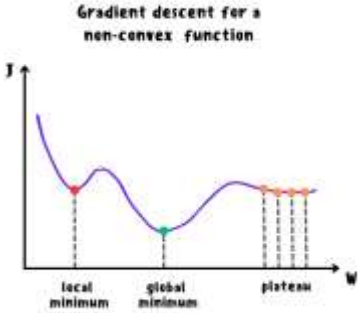
Where,

θ^* → model parameters (weights, biases, etc.)
J(θ) → loss function (how wrong the model is)
arg min → "the argument that minimizes"

Loss functions

- The loss functions can be divided into **convex** and **non-convex**.
- A function is **convex** if it has **one global minimum** and **no other local minima**.
- A function is **non-convex** if it has multiple local minima and maxima. Finding a global minimum is a **difficult task.**





Gradient decent

- Gradient Descent is an optimization algorithm used to minimize a cost (loss) function by iteratively updating parameters (weights).
- **In deep learning**, most popular optimization is done by ***Gradient Descent*** [Cauchy, 1847]
- Reduce the error by updating the parameters in the direction that **iteratively lowers the loss function**
- Requires taking the **derivative** of the loss function, which is an average of the losses computed on ***entire dataset***

Gradient descent variants

- There are three variants of gradient descent.
  - Batch gradient descent (BGD)
  - Stochastic gradient descent (SGD)
  - Mini-batch gradient descent (MGD)
- The difference of these algorithms is the **amount of data**.

This term is deferent with each method

$$\boxed{\theta := \theta - \alpha \nabla_{\theta} J(\theta)}$$

where, α = learning rate, $\nabla_{\theta} J(\theta)$ = gradient loss

Batch gradient descent

- This method computes the gradient of the cost function with the entire traning dataset
- The update rule is

$$\boxed{\theta := \theta - \alpha \nabla_{\theta} J(\theta)}$$

- Advantage
  - It is guaranteed to converge to the global minimum for convex error surfaces and to a local minimum for non-convex surfaces.
- Disadvantages
  - It can be very slow.
  - It is intractabie for datasets that do not fit in memory.
  - It does not allow us to update our mode online

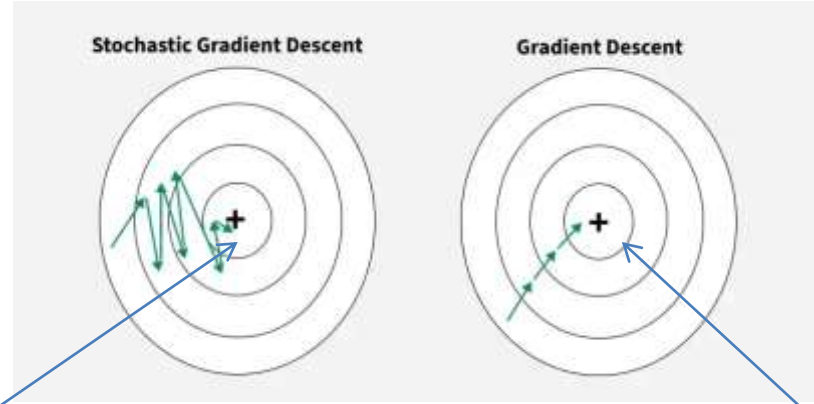
Stochastic gradient descent

- SGD, the parameter updates are made based on a single data point, not the entire dataset.
- The update rule is

$$\boxed{\theta := \theta - \alpha \nabla_{\theta} J(\theta)\,(x^{(i)}, y^{(i)})}$$

- **Advantage**
  - It is usually much faster than batch gradient descent.
  - It can be used to learn online.
- **Disadvantages**
  - It performs frequent updates with a high variance that cause the objective function to fluctuate

SGD vs BGD



SGD:

- The path is zig-zag, very noisy, and changes direction frequently.
- Steps are not smooth; they jump around.

BGD:

- The path is smooth and straight, moving consistently toward the center.
- The gradient direction is stable and consistent.

Minibatch Stochastic gradient

- Mini-Batch Stochastic Gradient Descent (Mini-Batch SGD) is a hybrid optimization method that combines the advantages of Batch Gradient Descent and Stochastic Gradient Descent (SGD).
- Mini-Batch SGD updates the model parameters using a small group (batch) of training samples instead of:
  - One sample → (SGD)
  - All samples → (BGD)

- A mini-batch usually contains 32, 64, 128, or 256 samples, depending on the dataset and model.
- The update rule is :

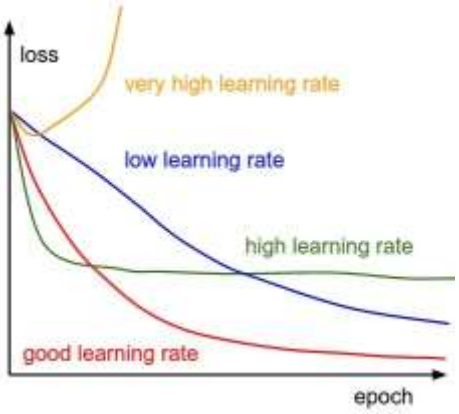
$$\boxed{\theta := \theta - \alpha \nabla_{\theta} J(\theta)\,(x^{(i:\,i\,+\,n)}, y^{(i:\,i\,+\,n)})}$$

Batch , stochastic and minibatch gradient

Type	Data per update	pros	cons
BGD	entire dataset	stable and smooth	very slow in large dataset
SGD	1 sample	fast update & flexiable	very noise
MiniBatchSGD	32-256	fast, stable	some tunning needed

Learning rate

- **Learning rate (LR)** is a hyperparameter in optimization algorithms (like Gradient Descent) that controls how big the step the model takes when updating its parameters.
- If learning rate is too small
    - Slow training
    - May get stuck in local minima
    - Needs many epochs



Learning rate (Con...)

- If learning rate is too large
  - Loss increases instead of decreasing
  - Model may diverge
  - Parameters overshoot the optimal point
- If learning rate is just right
  - Fast convergence
  - Smooth loss curve
  - Better performance

Techniques for Adjusting the Learning Rate

1. **Start with common default values**

   For many models:

   - Neural networks: 0.001
   - Adam optimizer: 0.001 (default)
   - SGD: 0.01

2. **Use Learning Rate Schedules**

   The best LR changes during training.

   Common schedules:

   - Step decay: reduce LR by factor every few epochs
   - Exponential decay
   - Cosine annealing
   - Warm-up + decay

Choose Learning Rate (Con..)

3. Use Learning Rate Finder

- You start with a very small Learning rate (like 1e-7) and increase exponentially until the loss blows up.
- Choose the learning rate where the loss decreases fastest..

4. Use adaptive optimizers

These automatically adjust LR for each parameter:

1. Adam
2. RMSProp
3. AdaGrad

5. **Experiment & observe loss curve**

- **Good learning rate:**

  Loss decreases smoothly

- **Bad learning rate:**

  Loss zig-zags or explodes upward

Gradient Descent with Momentum

- Gradient Descent with Momentum is an optimization algorithm that accelerates the convergence of standard gradient descent by adding a "velocity" term that accumulates past gradients.
    - Move faster in the correct direction
    - Reduce oscillations
    - Escape local minima
    - Converge much faster

Stochastic Gradient Descent **without** Momentum

Stochastic Gradient Descent **with** Momentum

- The update rule

$$v_t = \beta v_{t-1} + (1-\beta)\nabla_\theta J(\theta)$$
$$\theta := \theta - \alpha v_t$$

where vt is a velocity and β is a momentum cofficent (usually 0.9)

Nesterov Accelerated Gradient

- Nesterov Accelerated Gradient (NAG) is an improvement over momentum-based gradient descent that makes optimization faster and more stable.
- NAG is a variant of gradient descent that uses momentum, but with one smart change:
- In standard momentum, it compute the gradient at the current position. However, in Nesterov, it compute the gradient at the look-ahead position.
- This gives you a “heads-up” of the slope ahead — so you adjust early and avoid overshooting.
- The update rule

$$vt = \beta v_{t-1} + \alpha \nabla_\theta J(\theta - \beta v_{t-1})$$
$$\theta := \theta - v_t$$

where vt is a velocity and β is a momentum cofficent (usually 0.9)

AdaGrad (Adaptive Gradient)

- AdaGrad increases or decreases the learning rate per parameter depending on how often that parameter has been updated.
    - Parameters with frequent large gradients get smaller learning rates.
    - Parameters with small gradients get larger learning rates.
- Maintain a running squared gradient:

$$G_t = G_{t-1} + g_t^2$$

where, g_t is gradient at time t, G_t is accumulative gradient

Update rule:

$$\boxed{\theta := \theta - \frac{\alpha}{\sqrt{G_t + \epsilon}} g_t}$$

AdaGrad(Pron and cons)

Pros

- Good for sparse features
- Eliminates need to tune learning rate extensively

Cons

Accumulation of squared gradients grows indefinitely

- Learning rate becomes extremely small → training stops early

RMSProp (Root Mean Square Propagation)

- RMSProp was designed to fix AdaGrad's problem of shrinking learning rate too much.
- Instead of accumulating all gradients forever, RMSProp uses an exponential moving average (EMA) of past squared gradients.

$$E[g^2]_t = E[g^2]_t + (1-\beta)g^2{}_t$$

where,

- g_t is gradient at time t,
- $E[g^2]_t$ is updated value at time t,
- $E[g^2]_{t_1}$ is previous exponential average

Update rule:

$$\theta := \theta - \frac{\alpha}{\sqrt{E[g^2]_t + \epsilon}} g_t$$

Adam (Adaptive Moment Estimation)

- Adam (Adaptive Moment Estimation) optimizer combines the advantages of Momentum and RMSprop techniques to adjust learning rates during training.
- It works well with large datasets and complex models because it uses memory efficiently and adapts the learning rate for each parameter automatically.
- Adam builds upon two key concepts in optimization:
    1. Momentum

    2. RMSprop (Root Mean Square Propagation)

Con….

- **Adam** adapts the learning rate for each parameter individually by keeping track of:
- The First Moment (m_t) — the mean of gradients. This is like momentum, which smooths the gradient by averaging past gradients:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

- The Second Moment (v_t) — the uncentered variance of gradients. This is like RMSProp, which tracks the squared gradients:

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) {g_t}^2$$

Con....

- Becase of m_t and v_t starts zero, Adam corrects them:

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

- Parameter Update Rule

$$\theta := \theta - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

$\beta 1$ is 0.9 , $\beta 2$ is 0.999 , and ϵ is 1e−8